UNIT-3

Dr. Jisha P

SYLLABUS

Behavioral Modeling: Structured procedure, procedural assignments, timing control, conditional statements, multi-way branching, loops, sequential and parallel blocks, generate blocks, Examples.

BEHAVIORAL MODELING: STRUCTURED PROCEDURE

- There are two structured procedure statements in Verilog: **always and initial.**
- These statements are the two most basic statements in behavioral modeling.
- All other behavioral statements can appear only inside these structured procedure statements.

INITIAL STATEMENT

- All statements inside an **initial statement constitute an initial block**.
- An initial block starts at time 0, executes exactly once during a simulation, and then does not execute again.
- If there are multiple initial blocks, each block starts to execute concurrently at time 0.
- Each block finishes execution independently of other blocks.
- Multiple behavioral statements must be grouped, typically using the keywords **begin and end.** If there is only one behavioral statement, grouping is not necessary.

INITIAL STATEMENT - EXAMPLE

```
module stimulus;
reg x,y, a,b, m;
initial
m = 1'b0;
//single statement; does not need to
//be grouped
initial
begin
#5 a = 1'b1;
//multiple statements;
//need to be grouped
#25 b = 1'b0;
end
```

```
initial
begin
#10 x = 1'b0;
#25 y = 1'b1;
end
Initial
#50 $finish;
endmodule
```

INITIAL STATEMENT - EXAMPLE

```
time                    statement executed
0                       m = 1'b0;
5                       a = 1'b1;
10                      x = 1'b0;
30                      b = 1'b0;
35                      y = 1'b1;
50                      $finish;
```

INITIAL STATEMENT - EXAMPLE

- In the above example, the three initial statements start to execute in parallel at time 0.
- If a delay #<delay> is seen before a statement, the statement is executed <delay> time units after the current simulation time.
- Thus, the execution sequence of the statements inside the initial blocks will be as follows.
- The initial blocks are typically used for initialization, monitoring, waveforms and other processes that must be executed only once during the entire simulation run.
- The following subsections discussion how to initialize values using alternate shorthand syntax.
- The use of such shorthand syntax has the same effect as an initial block combined with a variable declaration.

COMBINED VARIABLE DECLARATION AND INITIALIZATION

Variables can be initialized when they are declared.

Example Initial Value Assignment

```
//The clock variable is defined first
reg clock;
//The value of clock is set to 0
initial clock = 0;
//Instead of the above method, clock variable can be initialized
at the time of //declaration This is allowed only for variables
declared at module level.
reg clock = 0;
```

COMBINED PORT/DATA DECLARATION AND INITIALIZATION

```
module adder (sum, co, a, b, ci);
output reg [7:0] sum = 0; //Initialize 8 bit output sum
output reg co = 0; //Initialize 1 bit output co
input [7:0] a, b;
input ci;
--
--
endmodule
```

COMBINED ANSI C PORT DECLARATION AND VARIABLE INITIALIZATION

```
module adder (output reg [7:0] sum = 0, //Initialize 8 bit output
output reg co = 0, //Initialize 1 bit output co
input [7:0] a, b,
input ci
);
--
--
endmodule
```

ALWAYS STATEMENT

- All behavioral statements inside an always statement constitute an always block.
- The always statement starts at time 0 and executes the statements in the always block continuously in a looping fashion.
- This statement is used to model a block of activity that is repeated continuously in a digital circuit.
- An example is a clock generator module that toggles the clock signal every half cycle.

```
module clock_gen (output reg
clock);
//Initialize clock at time zero
initial
clock = 1'b0;
//Toggle clock every half-cycle
//(time period = 20)
always
#10 clock = ~clock;
initial
#1000 $finish;
endmodule
```

PROCEDURAL ASSIGNMENTS

- Procedural assignments update values of reg, integer, real, or time variables.
- The value placed on a variable will remain unchanged until another procedural assignment updates the variable with a different value.
- Dataflow Modeling, where one assignment statement can cause the value of the right-hand-side expression to be continuously placed onto the left-hand-side net.
- The syntax for the simplest form of procedural assignment is shown below.

assignment ::= variable_lvalue = [delay_or_event_control] expression

PROCEDURAL ASSIGNMENTS

assignment ::= variable_lvalue = [delay_or_event_control] expression

- The left-hand side of a procedural assignment <lvalue> can be one of the following:
  - A reg, integer, real, or time register variable or a memory element
  - A bit select of these variables (e.g., addr[0])
  - A part select of these variables (e.g., addr[31:16])
  - A concatenation of any of the above
- The right-hand side can be any expression that evaluates to a value.
- There are two types of procedural assignment statements: blocking and nonblocking.

BLOCKING ASSIGNMENTS

= BLOCKING ASSIGNMENT

- Blocking assignment statements are executed in the order they are specified in a sequential block.
- A blocking assignment will block execution of statements that follow in a parallel block.
- **The = operator is used to specify blocking assignments.**

```
always @(posedge i_clock)
begin
  r_Test_1 = 1'b1;
  r_Test_2 = r_Test_1;
  r_Test_3 = r_Test_2;
end
```

EXAMPLE - BLOCKING STATEMENTS

```
reg x, y, z;
reg [15:0] reg_a, reg_b;
integer count;
//All behavioral statements must be inside an initial or always block
initial
begin
x = 0; y = 1; z = 1; //Scalar assignments
count = 0; //Assignment to integer variables
reg_a = 16'b0; reg_b = reg_a; //initialize vectors
#15 reg_a[2] = 1'b1; //Bit select assignment with delay
#10 reg_b[15:13] = {x, y, z} //Assign result of concatenation to part select of a vector
count = count + 1; //Assignment to an integer (increment)
end
```

EXAMPLE - BLOCKING STATEMENTS

The simulation times at which the statements are executed are as follows:

All statements x = 0 through reg_b = reg_a are executed at time = 0

- Statement reg_a[2] = 1 **at time = 15**
- Statement reg_b[15:13] = {x, y, z} **at time = 25**
- Statement count = count + 1 **at time = 25**
- **Since there is a delay of 15 and 10 in the preceding statements, count = count + 1 will be executed at time = 25 units**

NON-BLOCKING ASSIGNMENTS

<= NONBLOCKING ASSIGNMENT

- Nonblocking assignments allow scheduling of assignments without blocking execution of the statements that follow in a sequential block.
- A <= operator is used to specify nonblocking assignments.
- The operator <= is interpreted as a relational operator in an expression and as an assignment operator in the context of a nonblocking assignment.

Consider the following example where three concurrent data transfers take place at the positive edge of clock.

```
always @(posedge clock)
begin
reg1 <= #1 in1;
reg2 <= @(negedge clock) in2 ^ in3;
reg3 <= #1 reg1; //The old value of reg1
end
```

EXAMPLE - NONBLOCKING ASSIGNMENTS

```
reg x, y, z;
reg [15:0] reg_a, reg_b;
integer count; //All behavioral statements must be inside an initial or always block
initial
begin
x = 0; y = 1; z = 1; //Scalar assignments
count = 0; //Assignment to integer variables
reg_a = 16'b0; reg_b = reg_a; //Initialize vectors
reg_a[2] <= #15 1'b1; //Bit select assignment with delay
reg_b[15:13] <= #10 {x, y, z}; //Assign result of concatenation to part select of a vector
count <= count + 1; //Assignment to an integer (increment)
end
```

EXAMPLE - NONBLOCKING ASSIGNMENTS

In this example, the statements x = 0 through reg_b = reg_a are executed sequentially at time 0.

Then the three nonblocking assignments are processed at the same simulation

time.

1. reg_a[2] = 0 is scheduled to execute after 15 units **(i.e., time = 15)**
2. reg_b [15:13] = {x, y, z} is scheduled to execute after 10 time units

   (i.e.,time = 10)
3. count = count + 1 is scheduled to be executed without any delay

   (i.e., time = 0)

It is recommended that blocking and nonblocking assignments not be mixed in the same always block.!!

EXAMPLE - NONBLOCKING STATEMENTS TO ELIMINATE RACE CONDITIONS

```
//Illustration 1: Two concurrent always blocks with blocking statements
always @(posedge clock)
a = b;
always @(posedge clock)
b = a;
//Illustration 2: Two concurrent always blocks with nonblocking statements
always @(posedge clock)
a <= b;
always @(posedge clock)
b <= a;
```

Given below is an initial block with blocking procedural assignments. At what simulation time is each statement executed? What are the intermediate and final values of a, b, c, d?

```
initial
begin
    a = 1'b0;
    b = #10 1'b1;
    c = #5 1'b0;
    d = #20 {a, b, c};
end
```

ANSWER:

```
 0 a= 0, b= x, c= x, d= xxx
10 a= 0, b= 1, c= x, d= xxx
15 a= 0, b= 1, c= 0, d= xxx
35 a= 0, b= 1, c= 0, d= 010
```

Repeat exercise if nonblocking procedural assignments were used.

Answer:

```
 0 a= 0, b= x, c= x, d= xxx
 5 a= 0, b= x, c= 0, d= xxx
10 a= 0, b= 1, c= 0, d= xxx
```

What is the order of execution of statements in the following Verilog code? Is there any ambiguity in the order of execution? What are the final values of a,b,c,d?

Answer:

```
initial
begin
        a = 1'b0;
        #0 c = b;
end
initial
begin
        b = 1'b1;
        #0 d = a;
end
```

```
0 a= 0, b= 1, c= 1, d= 0
```

What is the final value of d in the following example?
(Hint: see intra-assignment delays.)

```
initial
begin
        b = 1'b1; c = 1'b0;
        #10 b = 1'b0;
initial
begin
        d = #25 (b | c);
end
```

Answer:

```
 0   b= 1, c= 0, d= x
10   b= 0, c= 0, d= x
25   b= 0, c= 0, d= 1
```

EXAMPLE - NONBLOCKING STATEMENTS TO ELIMINATE RACE CONDITIONS

- **Illustration 1: T**here is a race condition when blocking statements are used.
- Either a = b would be executed before b = a, or vice versa, depending on the simulator implementation.
- Thus, values of registers a and b will not be swapped.
- Instead, both registers will get the same value (previous value of a or b), based on the Verilog simulator implementation.
- **Illustration 2: E**liminate the race condition. At the positive edge of clock, the values of all right-hand-side variables are "read," and the right-hand-side expressions are evaluated and stored in temporary variables.
- During the write operation, the values stored in the temporary variables are assigned to the left-hand side variables.
- Separating the read and write operations ensures that the values of registers a and b are swapped correctly, regardless of the order in which the write operations are performed.

EXAMPLE - IMPLEMENTING NONBLOCKING ASSIGNMENTS USING BLOCKING ASSIGNMENTS

```
//Emulate the behavior of nonblocking assignments by using temporary variables and blocking assignments
always @(posedge clock)
begin
//Read operation - store values of right-hand-side expressions in temporary variables
temp_a = a;
temp_b = b;
//Write operation - Assign values of temporary variables to left-hand-side variables
a = temp_b;
b = temp_a;
end
```

Blocking Statements Vs Nonblocking Statements

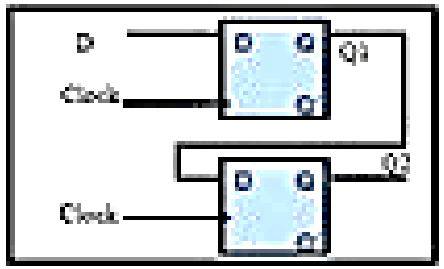
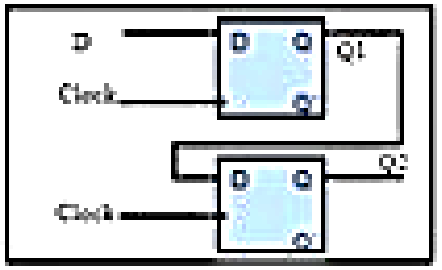
Blocking Assignments	Non Blocking Assignments
1.Represented using "=" operator sign between LHS & RHS	1. Represented using "<=" operator sign between LHS & RHS
2. Blocking assignments are used to represent Combinational logic	2. Non Blocking assignments are used to represent Sequential logic
3.Blocking assignments can be used in procedural assignments like initial, always and continuous assignments to nets like assign statements	3.Non Blocking assignments can be used in procedural assignments like initial, always and continuous assignments to nets like assign statement is not permitted.

4. Regular & Intra delay both are additive for Blocking assignments
`module example();`
`integer a = 0;`
`initial`
`begin`
`#2 a = #2 2;`
`b = #3 3;`
`end`
`endmodule`
At 4ns 'a' gets the value 2
At 7ns 'b' gets the value 3
	4. Regular delays are additive & intra delays are non additive in case of Non blocking assignments
	`module example();`
	`integer a = 0;`
	`initial`
	`begin`
	`#2 a <= #2 2;`
	`b <= #3 3;`
	`end`
	`endmodule`
	At 4ns 'a' gets the value 2
	At 5ns 'b' gets the value 3

Blocking Statements Vs Nonblocking Statements

Blocking Assignments	Non Blocking Assignments

5.Blocking assignment can have race condition.
 	
always@(posedge clock)
a= b;
always@(posedge clock)
b= a;
	5.Non-Blocking assignment can be used to avoid race condition
	
	always@(posedge clock)
	a <= b;
	always@(posedge clock)
	b <= a;

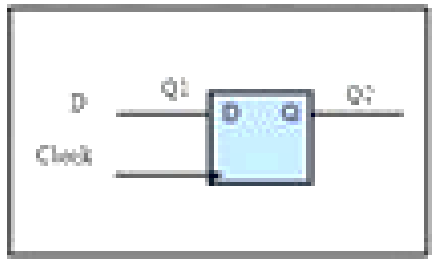
6.Blocking statements are Order dependent module non_blocking(input D,Clock, output reg Q2); reg Q1; always@(posedge Clock) begin Q2 = Q1; Q1 = D; end endmodule 	6.Non Blocking statements are Order Independent module non_blocking(input D,Clock, output reg Q2); reg Q1; always@(posedge Clock) begin Q1 <= D; Q2 <= Q1; end endmodule 

Blocking Statements Vs Nonblocking Statements

Blocking Assignments

7.These are executed sequentially, i.e. one statement is executed then the next statement is executed. One statement blocks the execution of other statements until it is executed. In the below example Q1 is trimmed off.

```
module blocking(input D,Clock,
                output reg Q2);

  reg Q1;
  always@(posedge Clock)
    begin
      Q1 = D;
      Q2 = Q1;
    end

endmodule
```

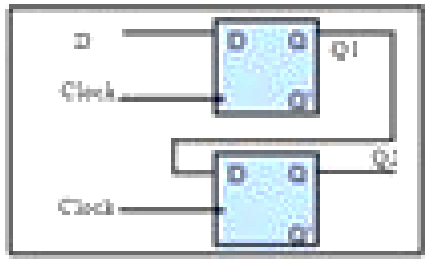


8. The Blocking assignments are updated in the Active region of the Stratified Event queue

Non Blocking Assignments

7.Its execution is concurrent with that of the following assignment or activity. For all the non-blocking assignments in a block, the right-hand sides are evaluated first. Subsequently the specified assignments are scheduled.

```
module non_blocking(input D,Clock,
                    output reg Q2);

  reg Q1;
  always@(posedge Clock)
    begin
      Q1 <= D;
      Q2 <= Q1;
    end

endmodule
```



8. The Non Blocking assignments RHS evaluated in the Active region & updated to LHS in Non Blocking region of the Stratified Event Queue.

TIMING CONTROLS

- Various behavioral timing control constructs are available in Verilog.
- In Verilog, if there are no timing control statements, the simulation time does not advance.
- Timing controls provide a way to specify the simulation time at which procedural statements will execute.
- There are three methods of timing control:

1. Delay-based timing control,
2. Event-based timing control
3. Level-sensitive timing control.

DELAY-BASED TIMING CONTROL

Delay-based timing control in an expression specifies the time duration between when the statement is encountered and when it is executed.

We used delay-based timing control statements when writing few modules in the preceding chapters but did not explain them in detail.

Delay-based timing control statements: Delays are specified by the symbol #.

DELAY-BASED TIMING CONTROL

Syntax for the delay-based timing control statement is shown below.

```
delay3 ::= # delay_value or # ( delay_value [ , delay_value [ ,
delay_value ] ] )
delay2 ::= # delay_value or # ( delay_value [ , delay_value ] )
```

```
delay_value ::=
unsigned_number
| parameter_identifier
| specparam_identifier
| mintypmax_expression
```

A delay_value can be any of the following:

1. **unsigned_number**: A non-negative numerical value (e.g., 5).
2. **parameter_identifier**: A parameter's name defined elsewhere.
3. **specparam_identifier**: A specific parameter name defined in the specparam section.
4. **mintypmax_expression**: An expression providing minimum, typical, and maximum values (e.g., (2:3:5)).

DELAY-BASED TIMING CONTROL

There are three types of delay control for procedural assignments:

- **Regular delay control**

  Regular delay control is used when a non-zero delay is specified to the left of a procedural assignment.

- **Intra-assignment delay control**

  Instead of specifying delay control to the left of the assignment, it is possible to assign a delay to the right of the assignment operator. Such delay specification alters the flow of activity in a different manner.

- **Zero delay control.**

  Zero delay control is a method to ensure that a statement is executed last, after all other statements in that simulation time are executed. This is used to eliminate race conditions. If there are multiple zero delay statements, the order between them is nondeterministic.

EXAMPLE - REGULAR DELAY CONTROL

```
//define parameters
parameter latency = 20;
parameter delta = 2;
reg x, y, z, p, q; //define register variables
initial
Begin
x = 0; // no delay control
#10 y = 1; // delay control with a number. Delay execution of  y = 1 by 10 units
#latency z = 0; // Delay control with identifier. Delay of 20 units
#(latency + delta) p = 1; // Delay control with expression
#y x = x + 1; // Delay control with identifier. Take value of y.
#(4:5:6) q = 0; // Minimum, typical and maximum delay
end
```

EXAMPLE - INTRA-ASSIGNMENT DELAYS

```
//define register variables
reg x, y, z;
initial //intra assignment delays
begin
x = 0; z = 0; //Take value of x and z at the time=0, evaluate x + z
y = #5 x + z; //then wait 5 time units to assign value to y.
end
//Equivalent method with temporary variables and regular delay control
initial
begin
x = 0; z = 0;
temp_xz = x + z; //Take value of x + z at the current time and
                 //store it in a temporary variable.

#5 y = temp_xz;     //Even though x and z might change between 0 and 5,the
                    //value assigned to y at time 5 is unaffected.

end
```

Intra-assignment delays are like using regular delays with a temporary variable to store the current value of a right-hand-side expression.

EXAMPLE - ZERO DELAY CONTROL

```
initial
begin
x = 0;
y = 0;
end
initial
begin
#0 x = 1; //zero delay control
#0 y = 1;
end
```

Using #0 is not a recommended practice.

EVENT-BASED TIMING CONTROL

An event is the change in the value on a register or a net.

Events can be utilized to trigger execution of a statement or a block of statements.

There are four types of event-based timing control:

1. Regular event control,
2. Named event control
3. Event OR control
4. Level-sensitive timing control.

1. REGULAR EVENT CONTROL

- The @ symbol is used to specify an event control.
- Statements can be executed on changes in signal value or at a positive or negative transition of the signal value.
- The keyword **posedge** is used for a positive transition
- **Example:**

```
@(clock) q = d; //q = d is executed whenever signal clock changes value

@(posedge clock) q = d; //q = d is executed whenever signal clock does a
                        //positive transition ( 0 to 1,x or z, x to 1, z to 1)

@(negedge clock) q = d; //q = d is executed whenever signal clock does a
                        //negative transition ( 1 to 0,x or z,x to 0, z to 0)

q = @(posedge clock) d; //d is evaluated immediately and assigned to q at
                        // the positive edge of clock
```

2. NAMED EVENT CONTROL

- Verilog provides the capability to declare an event and then trigger and recognize the occurrence of that event.
- The event does not hold any data. **A named event is declared by the keyword event. An event is triggered by the symbol ->.**
- The triggering of the event is recognized by the symbol @.
- **Example:**

```
    event received_data;
    always @(posedge clock)
    begin
    if(last_data_packet)
    ->received_data;
    end
always @(received_data)
data_buf = {data_pkt[0], data_pkt[1 data_pkt[2],data_pkt[3]};
```

3. EVENT OR CONTROL

- Sometimes a transition on any one of multiple signals or events can trigger the execution of a statement or a block of statements.
- This is expressed as an OR of events or signals.
- The list of events or signals expressed as an OR is also known as a sensitivity list.
- Sensitivity lists can also be specified using the "," (comma) operator instead of the or operator.
- The keyword **or** is used to specify multiple triggers.
- Two special symbols: **@* and @(*)** are sensitive to a change on any signal that may be read by the statement group that follows this symbol

Example:

```
//A level-sensitive latch with asynchronous reset
always @( reset or clock or d) //always @( reset, clock, d)
//Wait for reset or clock or d to change
begin
if (reset) //if reset signal is high, set q to 0.
q = 1'b0;
else if(clock) //if clock is high, latch input
q = d;
end
```

4. LEVEL-SENSITIVE TIMING CONTROL

- Event control discussed earlier waited for the change of a signal value or the triggering of an event.
- The symbol @ provided edge-sensitive control.
- Verilog also allows level sensitive timing control, that is, the ability to wait for a certain condition to be true before a statement or a block of statements is executed.
- The keyword wait is used for level sensitive constructs.
- **Example:**

```
always
wait (count_enable) #20 count = count + 1;
```

- In the above example, the value of count_enable is monitored continuously.
- If count_enable is 0, the statement is not entered.
- If it is logical 1, the statement count = count + 1 is executed after 20 time units.
- If count_enable stays at 1, count will be incremented every 20 time units

CONDITIONAL STATEMENTS

Conditional statements alter the flow within a behavior based upon certain conditions.

The choice among alternative statements depends on the Boolean value of an expression. The alternative statements can be a single statement or a block of statements delimited by the keywords begin . . . end.

The keywords **if and else** are used in conditional statements.

There are three categories of the conditional statement as shown below.

A true value is 1 or any nonzero value;

a false value is 0, x, or z.

If the evaluation is false, then the next expression in the activity flow is evaluated.

CONDITIONAL STATEMENTS

//nested if-else if //choice of multiple statements. One is executed.

```
if (expression1) statement1;        //if expression1 is true, then statement1 is executed.
else if (expression2) statement2;   //if expression2 is true, then statement2 is executed.
else if (expression3) statement3;   //if expression3 is true, then statement3 is executed.
else default statement;
```

CONDITIONAL STATEMENTS

- //Type 1 conditional statement. No else statement.
- //Statement executes or does not execute.
- if (<expression>) true_statement ;
- //Type 2 conditional statement. One else statement
- //Either true_statement or false_statement is evaluated
- if (<expression>) true_statement ; else false_statement ;
- //Type 3 conditional statement. Nested if-else-if.
- //Choice of multiple statements. Only one is executed.
- if (<expression1>) true_statement1 ;
- else if (<expression2>) true_statement2 ;
- else if (<expression3>) true_statement3 ;
- else default_statement ;

Eg: 4:1 MUX/ DEMUX

```
module multiplexer_4to1 (
    input wire [3:0] in,     // 4 data inputs
    input wire [1:0] sel,    // 2-bit select line
    output reg out           // Output
);
    always @(*)
    begin

        if (sel == 2'b00)
            out = in[0];
        else if (sel == 2'b01)
            out = in[1];
        else if (sel == 2'b10)
            out = in[2];
        else
            out = in[3];

    end
endmodule
```

```
module demultiplexer_1to4 (
    input wire in,           // Single data input
    input wire [1:0] sel,    // 2-bit select line
    output reg [3:0] out     // 4 outputs
);
    always @(*) begin

        out = 4'b0000; // Initialize all outputs to 0

        if (sel == 2'b00)
            out[0] = in;
        else if (sel == 2'b01)
            out[1] = in;
        else if (sel == 2'b10)
            out[2] = in;
        else
            out[3] = in;

    end
endmodule
```

EG: NESTED IF-ELSE – 4:1 MUX/DEMUX

```
module mux_4to1 (
    input wire [3:0] in,    // 4 data inputs
    input wire [1:0] sel,   // 2-bit select line
    output reg out   );     // Output

always @(*) begin
if (sel[1] == 0) begin
        if (sel[0] == 0) begin
            out = in[0];  // Select in[0]
        end
        else begin
            out = in[1];  // Select in[1]
        end
    end
else begin
        if (sel[0] == 0) begin
            out = in[2];  // Select in[2]
        end
        else begin
            out = in[3];  // Select in[3]
        end
    end
  end
endmodule
```

```
module demux_1to4 (
 input wire in,          // Single data input
 input wire [1:0] sel,   // 2-bit select line
 output reg [3:0] out );    // 4 outputs

always @(*) begin

    out = 4'b0000; // Initialize all outputs to 0

    if (sel[1] == 0)
        if (sel[0] == 0)
                out[0] = in;   // Route input to out[0]
        else
                out[1] = in;  // Route input to out[1]

    else
         if (sel[0] == 0)
                out[2] = in;  // Route input to out[2]
        else begin
                out[3] = in;  // Route input to out[3]
end
endmodule
```

MULTIWAY BRANCHING

In Verilog, multiway branching refers to implementing conditional control flow where multiple conditions or branches are evaluated, and only one branch is executed based on the condition that evaluates to true.

Verilog provides several constructs for multiway branching, such as

if-else if-else,

case, and

nested conditional operators (? :).

The case statement is a more concise way of implementing multiway branching when decisions are based on the value of a single variable or expression.

It is functionally similar to a "switch" statement in other programming languages.

Ideal when the decision depends on discrete, mutually exclusive values of a single variable.

1. CASE STATEMENT

- The keywords case, endcase, and default are used in the case statement.
- Each of statement1, statement2 , default_statement can be a single statement or a block of multiple statements.
- A block of multiple statements must be grouped by keywords begin and end.
- The default_statement is optional. Placing of multiple default statements in one case statement is not allowed.
- The case statements can be nested.

```
case (expression)
alternative1: statement1;
alternative2: statement2;
alternative3: statement3;
...
...
default:
default_statement;
endcase
```

CASE STATEMENT

```
//Execute statements based on the ALU control signal
reg [1:0] alu_control;
...
...
case (alu_control)
2'd0 : y = x + z;
2'd1 : y = x - z;
2'd2 : y = x * z;
default : $display("Invalid ALU control signal");
endcase
```

EG: CASE STATEMENT – 4:1 MUX

```
module mux_4_to_1( output reg out, input i0,i1,i2,i3,s1,s0 );
always @(i0,i1,i2,i3,s1,s0)
case ({s1,s0}) //{} concatenation
2'b00:out=i0;
2'b01:out=i1;
2'b10:out=i2;
2'b11:out=i3;
default:out=1'b0;
endcase
endmodule
```

EG: CASE STATEMENT – 1:4 DEMUX

```
module demux_1to4 (
    input wire in,          // Single data input
    input wire [1:0] sel,   // 2-bit select line
    output reg [3:0] out    );    // 4 output lines

    always @(*) begin           // always @(s1 or s0 or in)
      out = 4'b0000; // Initialize all outputs to 0

    case (sel)                  // Use case statement to select the appropriate output
          2'b00: out[0] = in;  // Route input to out[0]
          2'b01: out[1] = in;  // Route input to out[1]
          2'b10: out[2] = in;  // Route input to out[2]
          2'b11: out[3] = in;  // Route input to out[3]
          default: out = 4'b0000; // Default case (optional, for safety)
       endcase
    end
endmodule
```

EG: CASE STATEMENT - DECODER

```
module decoder(
input E,a2,a1,a0,
output reg [7:0]Y
);
always@(E,a2,a1,a0)
if(E==1)
case({a2,a1,a0})
3'b000:Y=8'd1;
3'b001:Y=8'd2;
3'b010:Y=8'd4;
3'b011:Y=8'd8;
```

```
3'b100:Y=8'd16;
3'b101:Y=8'd32;
3'b110:Y=8'd64;
3'b111:Y=8'd128;
default:Y=1'b0;
endcase
endmodule
```

CASE STATEMENT WITH X AND Z

The case statement compares 0, 1, x, and z values in the expression and the alternative bit for bit.

If the expression and the alternative are of unequal bit width, they are zero filled to match the bit width of the widest of the expression and the alternative.

- **casez** treats all z values in the case alternatives or the case expression as don't cares. All bit positions with z can also represented by ? in that position.
- **casex** treats all x and z values in the case item or the case expression as don't cares.

Account for unknown signals on select.

If any select signal is x then outputs are x.

If any select signal is z, outputs are z.

If one is x and the other is z, x gets higher priority.

CASE STATEMENT WITH X AND Z

```
always @(s1 or s0 or in)
case ({s1, s0}) //Switch based on control signals
2'b00 : begin out0 = in; out1 = 1'bz; out2 = 1'bz; out3
=1'bz; end
---
2'bx0, 2'bx1, 2'bxz, 2'bxx, 2'b0x, 2'b1x, 2'bzx :
begin
out0 = 1'bx; out1 = 1'bx; out2 = 1'bx; out3 = 1'bx;
end
2'bz0, 2'bz1, 2'bzz, 2'b0z, 2'b1z :
begin
out0 = 1'bz; out1 = 1'bz; out2 = 1'bz; out3 = 1'bz;
end
default: $display("Unspecified control signals");
endcase
endmodule
```

EG: CASE STATEMENT – ENCODER

```
module priority_encod(
input [7:0]I,
input e,
output reg[2:0]y
);
always @(I)
casex(I)
8'bXXXXXXX1:y=3'b000;
8'bXXXXXX10:y=3'b001;
8'bXXXXX100:y=3'b010;
8'bXXXX1000:y=3'b011;
8'bXXX10000:y=3'b100;
8'bXX100000:y=3'b101;
8'bX1000000:y=3'b110;
8'b10000000:y=3'b111;
endcase
endmodule
```

LOOPS

- There are four types of looping statements in Verilog: while, for, repeat, and forever.
- The syntax of these loops is very similar to the syntax of loops in the C programming language.
- All looping statements can appear only inside an initial or always block.
- Loops may contain delay expressions.
- Loop statements must be placed within an initial or an always block and may contain delay controls.
- The loop constructs allow for repeated execution of procedural statements within an initial or an always block

1. WHILE LOOP

- The while loop executes a procedural statement or a block of procedural statements as long as a Boolean expression returns a value of true.
- When the procedural statements are executed, the Boolean expression is re-evaluated.
- The loop is executed until the expression returns a value of false.
- If the evaluation of the expression is false, then the while loop is terminated and control is passed to the next statement in the module.
- If the expression is false before the loop is initially entered, then the while loop is never executed.
- The Boolean expression may contain any of the following types: arithmetic, logical, relational, equality, bitwise, reduction, shift, concatenation, replication, or conditional.
- If the while loop contains multiple procedural statements, then they are contained within the begin . . . end keywords. The syntax for a while statement is as follows:

while (expression)

procedural statement or block of procedural statements

EXAMPLE - WHILE LOOP

```
//Illustration 1: Increment count from 0 to 127. Exit at count 128.
//Display the count variable.
integer count;
initial
begin
count = 0;
while (count < 128) //Execute loop till count is 127 exit at count 128
begin
$display("Count = %d", count);
count = count + 1;
end
end
```

2. FOR LOOP

The for loop contains three parts:

1. An **initial** condition to assign a value to a register control variable. This is executed once at the beginning of the loop to initialize a register variable that controls the loop.
2. A **test condition** to determine when the loop terminates. This is an expression that is executed before the procedural statements of the loop to determine if the loop should execute. The loop is repeated as long as the expression is true. If the expression is false, the loop terminates and the activity flow proceeds to the next statement in the module.
3. An **assignment** to modify the control variable, usually an increment or a decrement. This assignment is executed after each execution of the loop and before the next test to terminate the loop

2. FOR LOOP

- The syntax of a for loop is shown below. The body of the loop can be a single procedural statement or a block of procedural statements.
- for (initial control variable assignment; test expression; control variable assignment)

  procedural statement or block of procedural statements
- The for loop is generally used when there is a known beginning and an end to a loop.
- The for loop is similar in function to the for loop in the C programming language and is used in a test bench.
- The initialization condition and the incrementing procedural assignment are included in the for loop and do not need to be specified separately. Thus, the for loop provides a more compact loop structure than the while loop. Note, however, that the while loop is more general-purpose than the for loop. The for loop cannot be used in place of the while loop in all situations.

EXAMPLE - FOR LOOP

```
integer count;
initial
for ( count=0; count < 128; count = count + 1)
$display("Count = %d", count);
```

```
//Initialize array elements
'define MAX_STATES 32
integer state [0: 'MAX_STATES-1]; //Integer array state with elements 0:31
integer i;
initial
begin
for(i = 0; i < 32; i = i + 2) //initialize all even locations with 0
state[i] = 0;
for(i = 1; i < 32; i = i + 2) //initialize all odd locations with 1
state[i] = 1;
end
```

3. REPEAT LOOP

The keyword repeat is used for this loop.

The repeat construct executes the loop a fixed number of times.

A repeat construct cannot be used to loop on a general logical expression.

A while loop is used for that purpose.

A repeat construct must contain a number, which can be a constant, a variable or a signal value.

However, if the number is a variable or signal value, it is evaluated only when the loop starts and not during the loop execution.

3. REPEAT LOOP

- The repeat loop executes a procedural statement or a block of procedural statements a specified number of times.
- The repeat construct can contain a constant, an expression, a variable, or a signed value.
- The syntax for the repeat loop is as follows:

  repeat (loop count expression)

  procedural statement or block of procedural statements

- If the loop count is x (unknown value) or z (high impedance), then the loop count is treated as zero. The value of the loop count expression is evaluated once at the beginning of the loop.

EXAMPLE - REPEAT LOOP

Two 8-bit registers are added to yield a sum of eight bits.

Register reg_a is initialized to a value of twenty-four; reg_b is initialized to a value of two.

The add operation is repeated eight times and register b is incremented by one after each add operation.

```
reg_a=00011000, reg_b=00000010, sum=00011010
reg_a=00011000, reg_b=00000011, sum=00011011
reg_a=00011000, reg_b=00000100, sum=00011100
reg_a=00011000, reg_b=00000101, sum=00011101
reg_a=00011000, reg_b=00000110, sum=00011110
reg_a=00011000, reg_b=00000111, sum=00011111
reg_a=00011000, reg_b=00001000, sum=00100000
reg_a=00011000, reg_b=00001001, sum=00100001
```

```
//example of the repeat keyword module
add_regs_repeat; reg [7:0] reg_a, reg_b, sum;

initial
begin
reg_a = 8'b0001_1000;
reg_b = 8'b0000_0010;
repeat (8)
begin
sum = reg_a + reg_b;
$display ("reg_a=%b, reg_b=%b, sum=%b",
reg_a, reg_b, sum);
reg_b = reg_b + 1;
end
end
endmodule
```

4. FOREVER LOOP

- The forever loop executes the procedural statement continuously until the system tasks $finish or $stop are encountered.
- It can also be terminated by the disable statement.
- The disable statement is a procedural statement; therefore, it must be used within an initial or an always block.
- It is used to prematurely terminate a block of procedural statements or a system task.
- When a disable statement is executed, control is transferred to the statement immediately following the procedural block or task.
- The forever loop is similar to a while loop in which the expression always evaluates to true (1).
- A timing control must be used with the forever loop; otherwise, the simulator would execute the procedural statement continuously without advancing the simulation time.

4. FOREVER LOOP

The syntax of the forever loop is as follows:

```
forever
        procedural statement
```

The forever statement is typically used for clock generation together with the system task $finish.

The loop does not contain any expression and executes forever until the $finish task is encountered.

EXAMPLE - FOREVER LOOP

```
//Example 1: Clock generation
//Use forever loop instead of always block
reg clock;
initial
begin
clock = 1'b0;
forever #10 clock = ~clock;
 //Clock with period of 20 units
end
```

```
//Example 2: Synchronize two register values at every positive edge of clock
reg clock;
reg x, y;
initial
forever @(posedge clock) x = y;
```

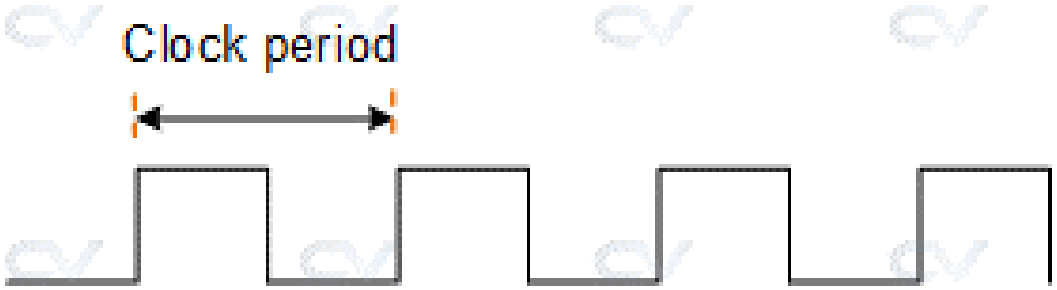
CLOCK GENERATION

Clocks are fundamental to building digital circuits as it allows different blocks to be in sync with each other.

Properties of a clock

The key properties of a digital clock are its ***frequency* which determines the *clock period*, its *duty cycle* and the *clock phase*** in relation to other clocks.

Clock Period: The frequency indicates how many cycles can be found in a certain period of time. And hence the clock period is the time taken to complete 1 cycle.



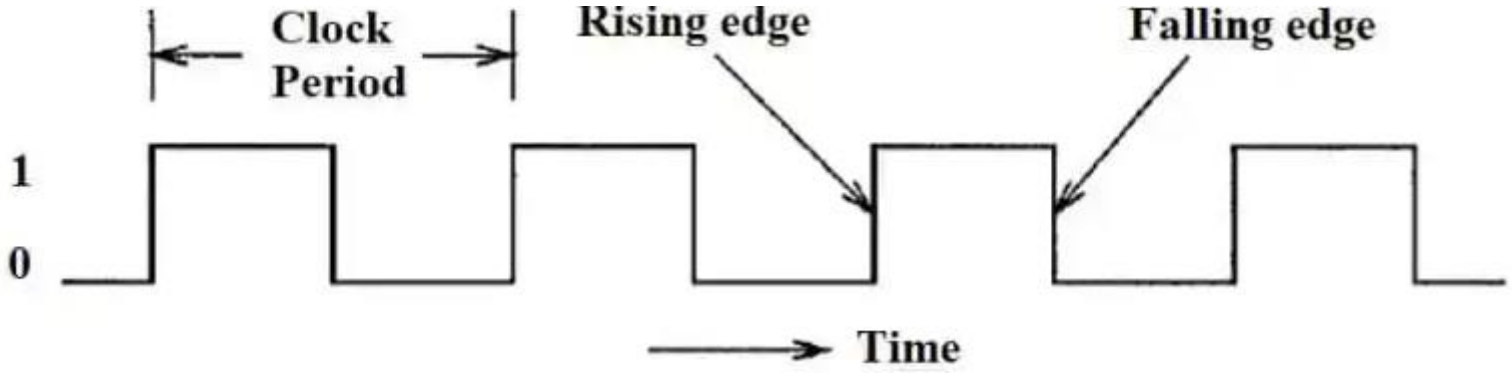
CLOCK GENERATION

1. Clock Period

Clock Period is the duration of one complete cycle of the clock signal.

It is the time between two successive rising or falling edges of the clock waveform.

Clock period is a fundamental parameter in digital systems as it determines the speed at which sequential logic operates.



The clock period, which is the reciprocal of the clock frequency, typically measures in time units like nanoseconds (ns), microseconds (μs), or milliseconds (ms).
The clock period affects how fast data can be processed in a digital system. A shorter clock period (higher frequency) means the system can process data more quickly, but it also requires faster circuitry and may introduce more timing challenges.

CLOCK GENERATION

The formula for calculating duty cycle is:
Duty Cycle = High Time/Pulse Period

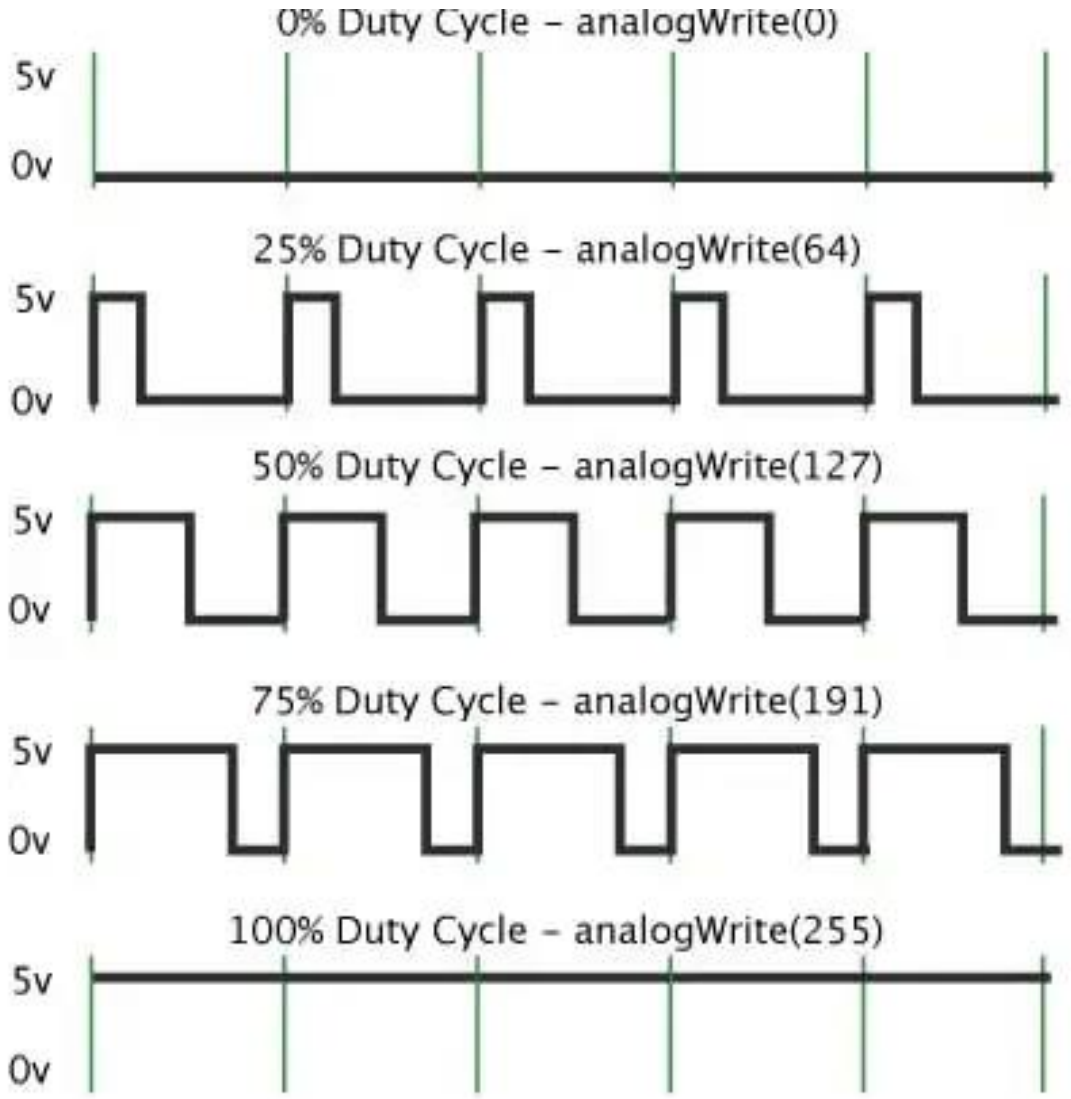
2. Clock Duty Cycle

Clock Duty Cycle represents the proportion of one clock period during which the clock signal is high (active) versus low (inactive). It is usually expressed as a percentage.

The duty cycle is the ratio of the time the clock signal is high to the total clock period.

A duty cycle of 50% means the clock signal is high and low for equal amounts of time, which is common in many digital systems.

A different duty cycle can affect the timing of data sampling and processing, especially in systems with asynchronous elements or those requiring specific timing constraints.

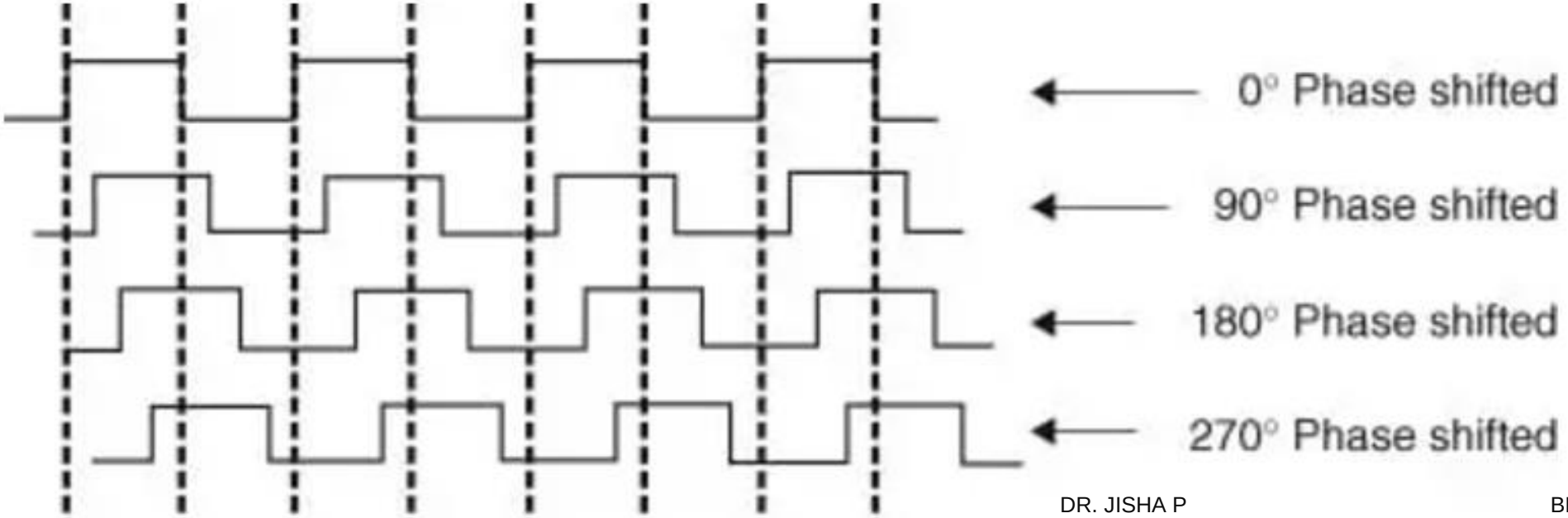


CLOCK GENERATION

3. Clock Phase

Clock Phase refers to the relative position of the clock signal with respect to a reference point or another clock signal. It describes the timing difference between two clock signals and is crucial in systems with multiple clocks or when synchronizing signals.

Clock phase indicates how one clock signal aligns with another in time. It is typically measured in degrees or time units.

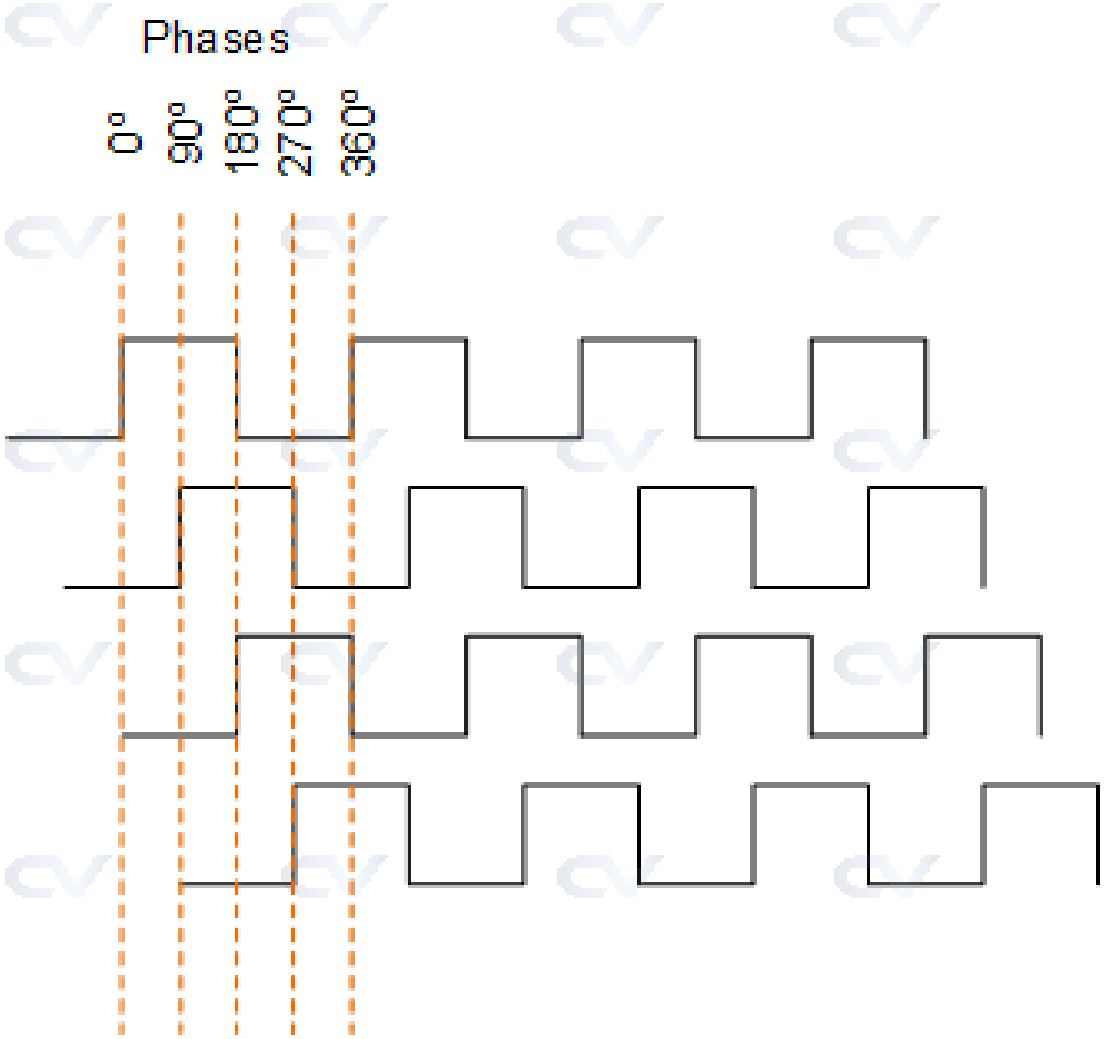


CLOCK GENERATION

If one cycle of a clock can be viewed as a complete circle with 360 deg, another clock can be relatively placed at a different place in the circle that occupy a different phase.

For example, another clock of the same time period that is shifted to the right by 1/4th of its period can be said to be at 90 deg in phase difference.

Proper clock phase alignment is essential for avoiding timing issues such as setup and hold violations in synchronous designs. In systems with multiple clock domains, phase synchronization ensures that data is correctly sampled and transferred between different clock domains.



CLOCK GENERATION

In Verilog, a clock generator is a module or block of code that produces clock signals for digital simulations and designs.

These clock signals drive sequential logic elements, such as flip-flops and registers, ensuring that data processes in sync with the design's timing requirements.

A clock generator in Verilog serves the following purposes:

Produces Clock Signals: It generates periodic clock pulses that can be used to drive various components in your design. These pulses create a regular timing rhythm that sequential elements depend on to operate correctly.

Defines Clock Characteristics: The clock generator specifies the frequency and duty cycle of the clock signal. Frequency determines how fast the clock ticks, while the duty cycle defines the proportion of time the clock signal is high versus low.

Provides Timing for Simulations: In testbenches, a clock generator is often used to create a clock signal that drives the simulation. This allows you to observe how your design behaves over time and under different clock conditions.

CLOCK GENERATION- EXAMPLE USING FOREVER

```
module clock_generator( output reg clk );
// Define clock period in simulation time units
parameter CLOCK_PERIOD = 10;
// Clock period in simulation time units
// Initial block to generate clock pulses
initial
begin
clk = 0; // Initialize clock to 0
forever # (CLOCK_PERIOD / 2) clk = ~clk;
// Toggle clock every half period end endmodule
```

CLOCK GENERATION- EXAMPLE USING ALWAYS

```
module clock_generator ( output reg clk );
// Parameter to define the clock period in time units
parameter PERIOD = 10; // Clock period in time units
initial     // Initial block to set the initial state of the clock
begin
clk = 0; // Initialize the clock signal to 0
end
always // Always block to toggle the clock signal
begin
# (PERIOD / 2) clk = ~clk; // Toggle the clock every half period
end
endmodule
```

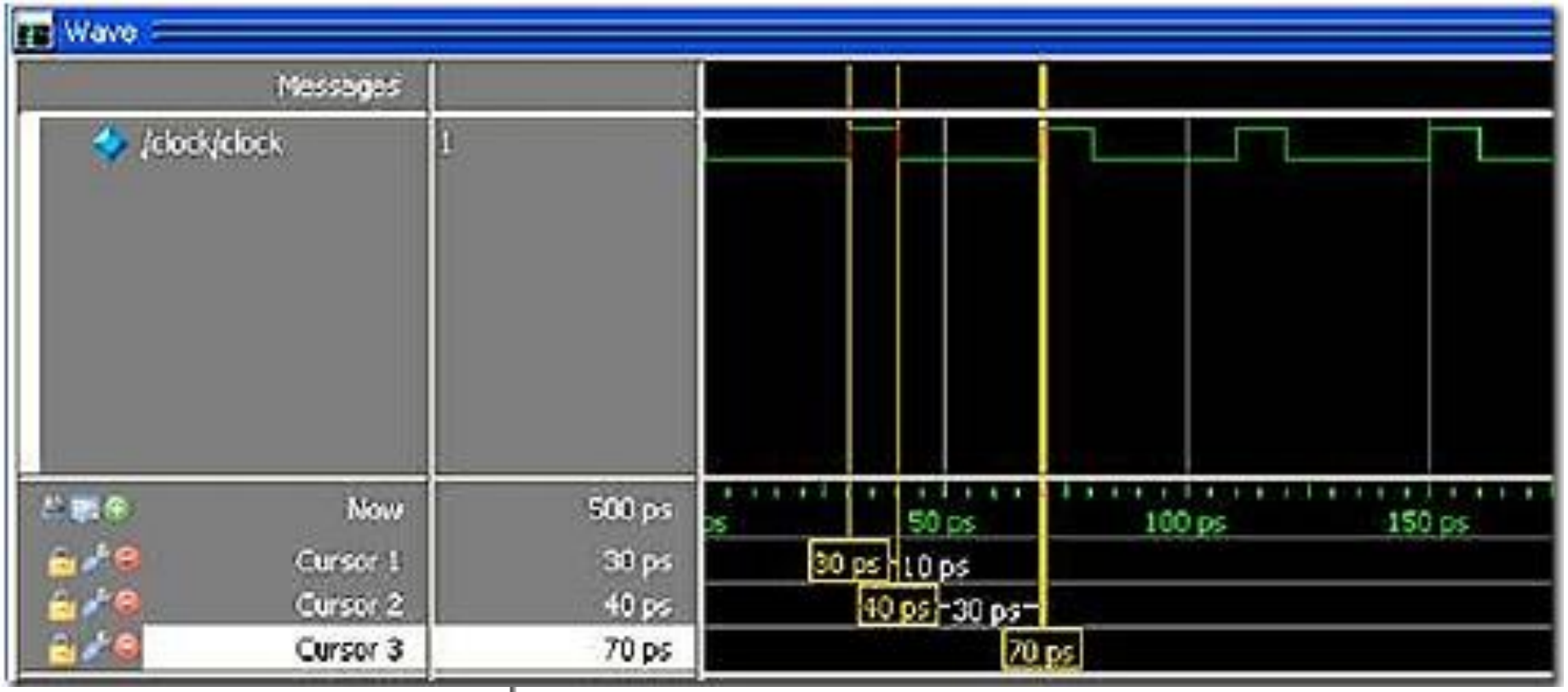
CLOCK GENERATION WITH VARIABLE DUTY CYCLE

```
module clk_gen (output  clk) ;
parameter CLK_PERIOD = 10;
parameter DUTY_CYCLE = 60; //60% duty cycle parameter
TCLK_HI = (CLK_PERIOD*DUTY_CYCLE/100);
parameter TCLK_LO = (CLK_PERIOD-TCLK_HI);
reg clk;
initial clk = 0;
```

```
always
begin

#TCLK_LO;
clk = 1'b1;
#TCLK_HI;
clk = 1'b0;

end
endmodule
```

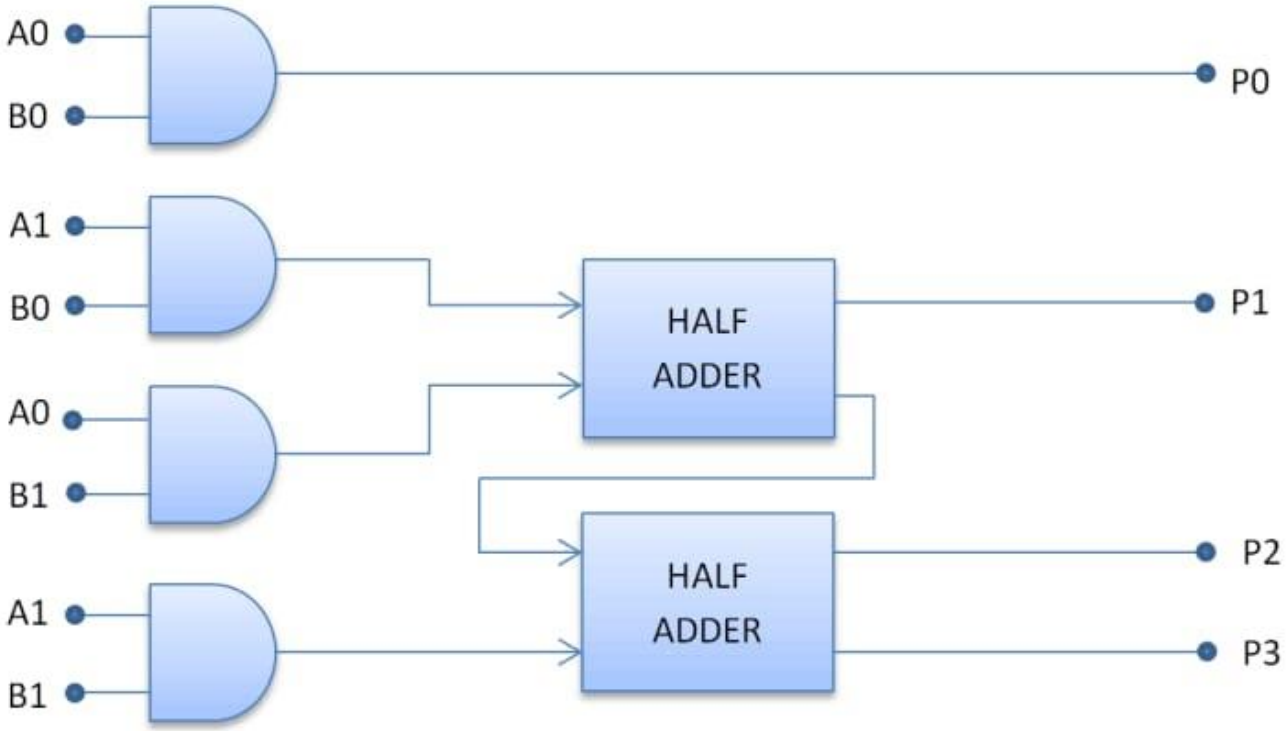
Design a clock with time period = 40ns and a duty cycle of 25% by using the always and initial statements.
The value of clock at time = 0 should be initialized to 0.

```
//ex7_2 clock
module clock;
reg clock;

initial
  clock=1'b0;

always
begin
  #30 clock=~clock;
  #10 clock=~clock;
end

initial
  $monitor($time, " clock= %b", clock);

endmodule
```



2 Bit Array Multiplier- Dataflow Modelling

		b1	b0
		a1	a0
		a0b1	a0b0
	a1b1	a1b0	
c3	c2	c1	c0



2 Bit Array Multiplier – gate level

```
module multiplier (a,b, y);
   input [1:0] a, b;
   output [3:0] y;
        wire t1, t2, t3,t4;
        and a1(y[0],a[0],b[0]);
        and a2 (t1,a[1],b[0]);
        and a3(t2,a[0],b[1]);
        and a4(t3,a[1],b[1]);
        halfadder ha0(t1,t2,y[1],t4);
        halfadder ha1(t3,t4,y[2],y[3]);
endmodule
```

```
module halfadder(a,b, s,co);
        input a,b;
        output s,co;
         xor x1(s,a,b);
         and a1(co,a,b);
endmodule
```

```
module array_multiplier_2bit (
    input [1:0] A, // 2-bit multiplicand
    input [1:0] B, // 2-bit multiplier
    output [3:0] P // 4-bit product );
    wire p0, p1, p2, p3, p4;   // Partial products
    // Partial product generation
    assign p0 = A[0] & B[0]; // Least significant bit of product
    assign p1 = A[1] & B[0];
    assign p2 = A[0] & B[1];
    assign p3 = A[1] & B[1]; // Most significant bit of product
    // Summing partial products
    assign P[0] = p0; // LSB of result
    assign P[1] = p1 ^ p2; // Adding middle bits
    assign p4 = p1 & p2; // Carry from the middle bits
    assign P[2] = p3 ^ p4; // Adding carry to the next bit
    assign P[3] = p3 & p4; // MSB of result

endmodule
```

			a_1a_0
		$\times$	b_1b_0
		a_1b_0	a_0b_0
	a_1b_1	a_0b_1	
p_3	p_2	p_1	p_0

```
module testbench;
    reg [1:0] A, B; // 2-bit inputs
    wire [3:0] P;   // 4-bit product

    // Instantiate the multiplier
    array_multiplier_2bit uut (  .A(A),  .B(B), .P(P)   );

    initial begin
        $monitor("A = %b, B = %b, P = %b", A, B, P);
        A = 2'b00; B = 2'b00; #10; // Test all possible combinations
        A = 2'b01; B = 2'b01; #10;
        A = 2'b10; B = 2'b01; #10;
        A = 2'b11; B = 2'b10; #10;
        A = 2'b11; B = 2'b11; #10;

        $finish;
    end
endmodule
```

SEQUENTIAL AND PARALLEL BLOCKS

Procedural flow control statements modify the flow in a behavior by selecting branch options, repeating certain activities, selecting a parallel activity, or terminating an activity.

The activity can occur in sequential blocks or in parallel blocks.

Block statements are used to group multiple statements to act together as one.

In previous examples, we used keywords begin and end to group multiple statements.

Thus, we used sequential blocks where the statements in the block execute one after another.

SEQUENTIAL BLOCKS

Block Types

There are two types of blocks: **sequential blocks and parallel blocks.**

Sequential blocks

The keywords **begin and end** are used to group statements into sequential blocks.

Sequential blocks have the following characteristics:

- The statements in a sequential block are processed in the order they are specified. A statement is executed only after its preceding statement completes execution (except for nonblocking assignments with intra-assignment timing control).
- If delay or event control is specified, it is relative to the simulation time when the previous statement in the block completed execution.

Example - Sequential Blocks

```
//Illustration 1: Sequential block without delay
reg x, y;
reg [1:0] z, w;
initial
begin
x = 1'b0;
y = 1'b1;
z = {x, y};
w = {y, x};
end
```

```
//Illustration 2: Sequential blocks with delay.
reg x, y;
reg [1:0] z, w;
initial
begin
x = 1'b0; //completes at simulation time 0
#5 y = 1'b1; //completes at simulation time 5
#10 z = {x, y}; //completes at simulation time 15
#20 w = {y, x}; //completes at simulation time 35
end
```

PARALLEL BLOCKS

Parallel blocks, specified by keywords fork and join, provide interesting simulation features.

Parallel blocks have the following characteristics:

- Statements in a parallel block are executed concurrently.
- Ordering of statements is controlled by the delay or event control assigned to each statement.
- If delay or event control is specified, it is relative to the time the block was entered.

All statements in a parallel block start at the time when the block was entered. Thus, the order in which the statements are written in the block is not important.

Example - Parallel Blocks

```
//Example 1: Parallel blocks with delay.
reg x, y;
reg [1:0] z, w;
initial
fork
x = 1'b0; //completes at simulation time 0
#5 y = 1'b1; //completes at simulation time 5
#10 z = {x, y}; //completes at simulation time 10
#20 w = {y, x}; //completes at simulation time 20
join
```

Special Features of Blocks

Three special features available with block statements: **nested blocks, named blocks, and disabling of named blocks.**

Nested blocks

Blocks can be nested. Sequential and parallel blocks can be mixed.

Example - Nested Blocks

```
//Nested blocks
initial
begin
x = 1'b0;
fork
#5 y = 1'b1;
#10 z = {x, y};
join
#20 w = {y, x};
end
```

Special Features of Blocks

Named blocks

Blocks can be given names.

- Local variables can be declared for the named block.
- Named blocks are a part of the design hierarchy. Variables in a named block can be accessed by using hierarchical name referencing.
- Named blocks can be disabled, i.e., their execution can be stopped.

Example 7-29 Named Blocks

```
//Named blocks
module top;
initial
begin: block1 //sequential block named block1
integer i; //integer i is static and local to
block1 can be accessed by hierarchical name,
top.block1.i
...
end
initial
fork: block2 //parallel block named block2
reg i; // register i is static and local to
block2 can be accessed by hierarchical name,
top.block2.i
...
...
join
```

Special Features f Blocks

The keyword **disable** provides a way to terminate the execution of a named block.

Disable can be used to get out of loops, handle error conditions, or control execution of pieces of code, based on a control signal.

Disabling a block causes the execution control to be passed to the statement immediately succeeding the block.

For C programmers, this is very similar to the break statement used to exit a loop.

The difference is that a break statement can break the current loop only, whereas the keyword disable allows disabling of any named block in the design.

Example - disabling named blocks

```
//Illustration: Find the first bit with a value 1 in flag (vector variable)
reg [15:0] flag;
integer i; //integer to keep count
initial
begin
flag = 16'b 0010_0000_0000_0000;
i = 0;
begin: block1 //The main block inside while is named block1
while(i < 16)
begin
if (flag[i])
begin
$display("Encountered a TRUE bit at element number %d", i);
disable block1; //disable block1 because you found true bit.
end
i = i + 1;
end
end
end3
```

Procedural Blocks: Begin-end and Fork-join

begin-end and fork-join are used to combine a group of statements in a single block.

General syntax with begin-end is as follows:

```
type_of_block @(sensitivity_list)
begin: group_name
local_variable_declarations;
statements;
end
```

type_of_block may be initial or always. sensitivity_list is optional and possible only in always block.

Sensitivity list is used to model sequential or combinational logic.

Eg:

```
module comb(a, b, c);
    input c;
    input b;
    output a;

    reg a;

always @( c or b)
begin
    a = c & b;
end

endmodule
```

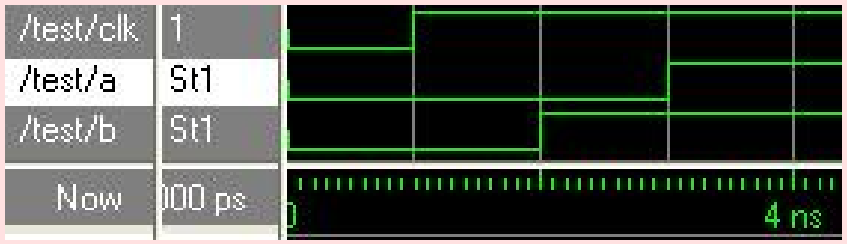
```
module seq(a, b, c);
    output a;
    input b;
    input c;

    reg a;

always @( posedge c)
begin
    a <= b;
end

endmodule
```

PROCEDURAL BLOCKS: BEGIN-END AND FORK-JOIN

- Inside an initial or always block, we can group statements using begin--end or fork--join.
- **begin--end groups two or more statements together sequentially, so that statements are evaluated in the order they are listed.**
- Each timing control is relative to the previous statement.
- **fork--join groups two or more statements together in parallel, so that all statements are evaluated concurrently.**
- Each timing control is absolute to when the group started.

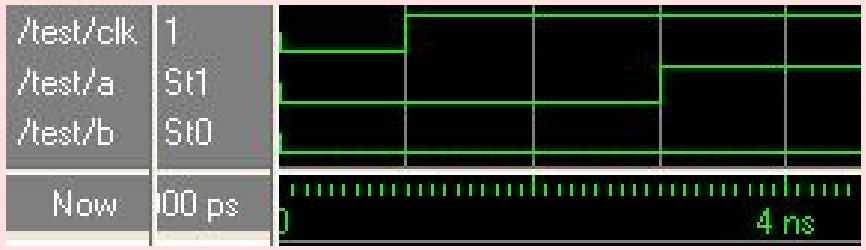
Begin--end and Fork--join Execution.

```
module forkjoin(clk, a, b);
    input clk;
    output a;
    output b;

    reg a, b;

    initial
    begin
        a = 0;
        b = 0;
    end

    always @(posedge clk)
    fork
        #2 a = 1;
        #1 b = 1;
    join
endmodule
```



Output of forkjoin ^

```
module forkjoin1(clk, a, b);
    input clk;
    output a;
    output b;

    reg a, b;

    initial
    begin
        a = 0;
        b = 0;
    end

    always @(posedge clk)
    fork
        #2 a = 1;
        #1 b = a;
    join
endmodule
```



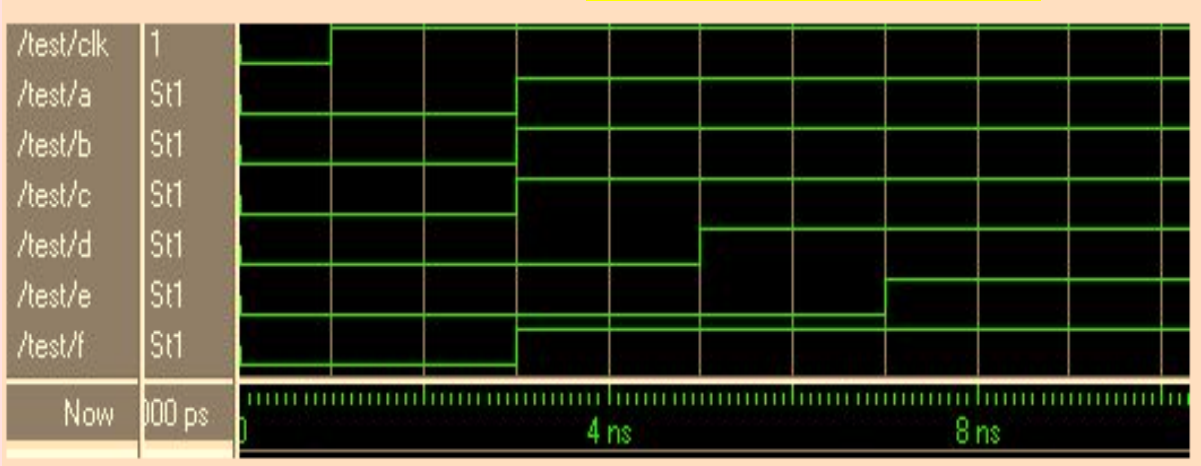
Output of forkjoin1 ^

Begin--end and Fork--join Execution.

- We can nest begin--end and fork--join.
- That is, inside a begin--end we can have fork--join and inside fork--join we can have begin--end.
- Consider these codes to find out how nested begin--end and fork--join works.

```
module nesting1(clk, a, b, c, d, e, f);
    input clk;
    output a, b, c, d, e, f;

    reg a, b, c, d, e, f;

    initial
    begin
       a = 0;
       b = 0;
       c = 0;
       d = 0;
       e = 0;
       f = 0;
    end

    always @(posedge clk)
    fork
        #2 a = 1;
        #2 b = 1;
        begin
            #2 c = 1;
            #2 d = 1;
            #2 e = 1;
        end
        #2 f = 1;
    join
endmodule
```

OUTPUT



EG: Analyse the code and find value of variables

```
(i) reg [3:0] temp = 4'd0,
a = 4'd1, b=4'd2;

initial begin

temp <= a;

a <= b;

b <= a;

end
```

Initial Values: temp = 4'd0, a = 4'd1, b = 4'd2

After scheduling the non-blocking assignments:

- **temp <= a** schedules temp to take the value of a **(4'd1).**
- **a <= b** schedules a to take the value of b **(4'd2).**
- **b <= a** schedules b to take the value of a (**4'd1**, its original value at the start of this block).

Once all the statements are processed at the end of the current time step:

- temp = 4'd1 (value of a at the start of the block).
- a = 4'd2 (value of b at the start of the block).
- b = 4'd1 (value of a at the start of the block).

Final Values: **temp = 4'd1, a = 4'd2, b = 4'd1.**

EG: Analyse the code and find value of variables

(ii)

```
reg [3:0] a = 4'd1,
b=4'd2;

initial

begin

a = b;

b = a;

end
```

Initial Values: a = 4'd1, b = 4'd2

Execution of statements:

a = b: Assigns the value of b (4'd2) to a.

Now, a = 4'd2.

b = a: Assigns the updated value of a (4'd2) to b.

Now, b = 4'd2.

Final Values: a = 4'd2, b = 4'd2.